

RESUMEN BASE DE DATOS

CONCEPTO DE BASES DE DATOS:

Una base de datos es una colección de datos interrelacionados almacenados en conjunto, sin redundancias perjudiciales o innecesarias; su finalidad es servir a una aplicación o mas, de la mejor manera posible.

Los datos se almacenan de forma que resulten independientes de los programas que los usan, además se emplean métodos bien determinados para incluir nuevos datos y para modificar o extraer los datos almacenados.

SISTEMA DE ADMINISTRACIÓN DE DATOS (DBMS. DATABASE MANAGEMENT SYSTEM)

Nota: El DBMS es el motor de la base de datos, el sistema de base de datos está formado tanto por el motor como por la base de datos (hardware, software, usuarios y datos)

Un sistema de administración de datos es una colección de programas que permiten a los usuarios crear y mantener una base de datos y a las aplicaciones interaccionar con la BD. Sirve como interfaz entre los usuarios y los datos y se encarga de ocultar la complejidades de la implementación de una BD

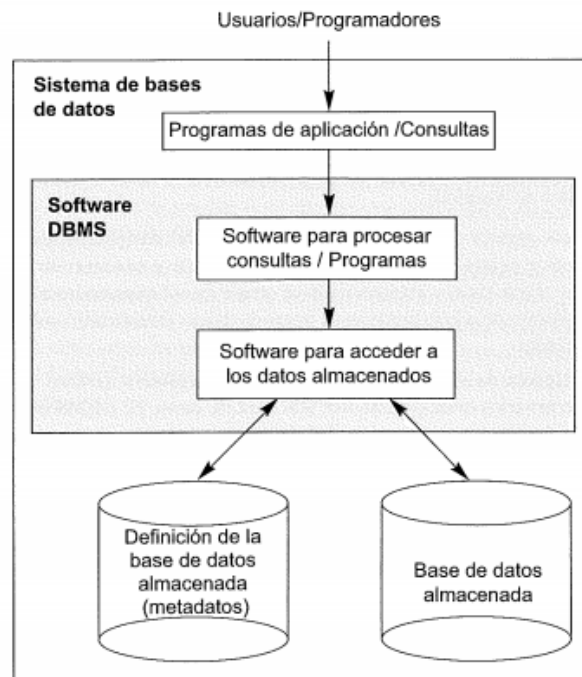
El DBMS es un sistema de software de propósito general que facilita los procesos de definición, construcción, manipulación y compartición de bases de datos entre varios usuarios y aplicaciones.

El DBMS permite:

1. Definición: Esto implica especificar los tipos de datos, estructuras y restricciones de los datos que se almacenarán en la base de datos. La definición de una base de datos también se guarda en la misma en forma de diccionario de la base de datos mejor conocida como metadatos. Utiliza DDL (create, drop, alter)
2. Manipulación: Incluye funciones como la consulta de la base de datos para recuperar datos específicos, actualizar la base de datos y generar informes a partir de los datos. Utiliza DML(insert, delete, update, select)
3. Seguridad: Permite dar y revocar permisos. Utiliza DCL (grant, revoke)
4. Integridad de los datos:
5. Respaldo y recuperación de Bases de Datos:
6. Transacciones: No permite guardar transacciones a medias.

Otra de las funciones ofrecidas por el DBMS son la protección del sistema contra el funcionamiento defectuoso del hardware o el software (caídas) y la protección de la seguridad contra el acceso no autorizado o malintencionado.

Un sistema de bases de datos es la combinación de una base de datos y software DBMS.



FUNCIONES DEL DBMS

- Controlar la seguridad.
- Hacer cumplir las reglas de integridad.
- Permitir el acceso concurrente.
- Permitir copia de seguridad y recuperación.
- Suministrar diferentes vistas.
- Permitir crear estructuras de almacenamiento para procesamiento eficaz.
- Facilitar carga de datos desde otros archivos
- Permitir reorganizar la BD para optimizar.
- Mantener estadísticas y permitir el monitoreo

Comparación:

Sistemas tradicionales de ficheros	Sistemas de bases de datos
• Sist. orientados a los procesos	✓ Sist. orientados a los datos (uso racional)
• Ficheros específicos para aplicaciones	✓ No son diseñados para una aplicación
• Redundancia de datos	✓ Disminuir redundancia
• Inconsistencia de datos	✓ Evitar inconsistencia
• Pérdida de espacio físico	✓ Mantener formato de datos
• Dependencia física de datos	✓ Independencia física
• Dependencia lógica de datos	✓ Independencia lógica
	✓ Compartir los datos
	✓ Aplicar restricciones de seguridad
	✓ Mantener la integridad
	- Instalación costosa
	- Baja estandarización
	- Personal especializado

COMPONENTES DE UN SISTEMA DE BASE DE DATOS:

1. Hardware
2. Usuarios
3. Software
4. Datos

USUARIOS DE UN SISTEMA DE BASE DE DATOS:

Analista de Sistemas: Los analistas de sistemas determinan los requisitos de los usuarios finales, especialmente de los usuarios finales principiantes y paramétricos, así como las especificaciones de desarrollo para las transacciones enlatadas que satisfacen esos requisitos.

Diseñador de BD: Los diseñadores de las bases de datos son los responsables de identificar los datos que se almacenarán en la base de datos y de elegir las estructuras apropiadas para representar y almacenar esos datos.

DBA: El DBA es responsable del acceso autorizado a la base de datos, de la coordinación y monitorización de su uso, y de adquirir los recursos software y hardware necesarios. El DBA también es responsable de problemas como las brechas de seguridad o de unos tiempos de respuesta pobres.

Posee otras funciones como:

- Definir estructuras
- Definir estructuras físicas
- ESpecificar perfiles y seguridad
- IMplementar integridad
- Definir métodos de respaldo y recuperación
- Monitorear el rendimiento

Programadores de aplicaciones: Los programadores de aplicaciones implementan las especificaciones, aportadas por los analistas de sistemas, como programas; después, verifican, depuran, documentan y mantienen esas transacciones enlatadas.

Usuario Final: Los usuarios finales son las personas cuyos trabajos requieren acceso a la base de datos para realizar consultas, actualizaciones e informes.

Se clasifican en:

- Usuarios casuales: acceden ocasionalmente a la base de datos, pero pueden necesitar una información diferente en cada momento. Utilizan un sofisticado lenguaje de consulta de bases de datos para especificar sus peticiones
- Usuarios principiantes o paramétricos: Su labor principal gira en torno a la consulta y actualización constantes de la base de datos, utilizando tipos de consultas y actualizaciones estándar (denominadas transacciones enlatadas) que se han programado y probado cuidadosamente. Ej: Cajero bancario.
- Usuarios sofisticados: entre estos se encuentran ingenieros, científicos, analistas comerciales y otros muchos que están completamente familiarizados con el DBMS a fin de implementar sus aplicaciones y satisfacer sus complejos requisitos.

- Usuarios independientes: estos mantienen bases de datos personales utilizando paquetes de programas confeccionados que proporcionan unas interfaces fáciles de usar y basadas en menús o gráficos.

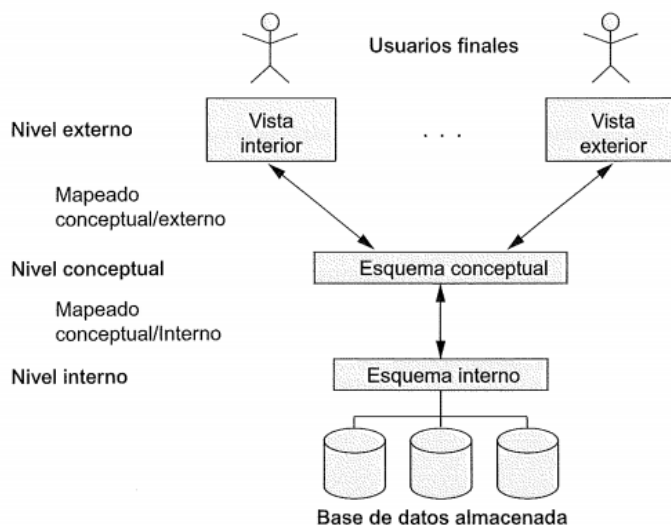
Implementador de DBMS: Diseñan e implementan los módulos y las interfaces DBMS como un paquete software.

Implementador de Herramientas: Diseñan e implementan herramientas. Las herramientas son paquetes opcionales que a menudo se compran por separado. Entre ellas podemos citar los paquetes para el diseño de bases de datos, la monitorización del rendimiento, las interfaces gráficas o en otros idiomas, el prototipado, la simulación y la generación de datos de prueba.

Personal de mantenimiento: Son los responsables de la ejecución y el mantenimiento real del entorno hardware y software para el sistema de bases de datos.

ARQUITECTURA DE TRES ESQUEMAS

El objetivo de la arquitectura de tres esquemas es separar las aplicaciones de usuario y las bases de datos físicas.



Nivel interno: Tiene un esquema interno, que describe la estructura de almacenamiento físico de la base de datos. El esquema interno utiliza un modelo de datos físico y describe todos los detalles del almacenamiento de datos y rutas de acceso a la base de datos.

Nivel conceptual: Tiene un esquema conceptual que describe la estructura de toda la base de datos para una comunidad de usuarios. Este esquema oculta los detalles de las estructuras de almacenamiento físico y se concentra en describir entidades, los tipos de datos, las relaciones, las operaciones de los usuarios y las restricciones. (se basa en un modelo de datos de alto nivel)

Nivel de vista o externo: incluye una cierta cantidad de esquemas externos o vistas de usuario. Un esquema externo describe la parte de la base de datos en la que un grupo de

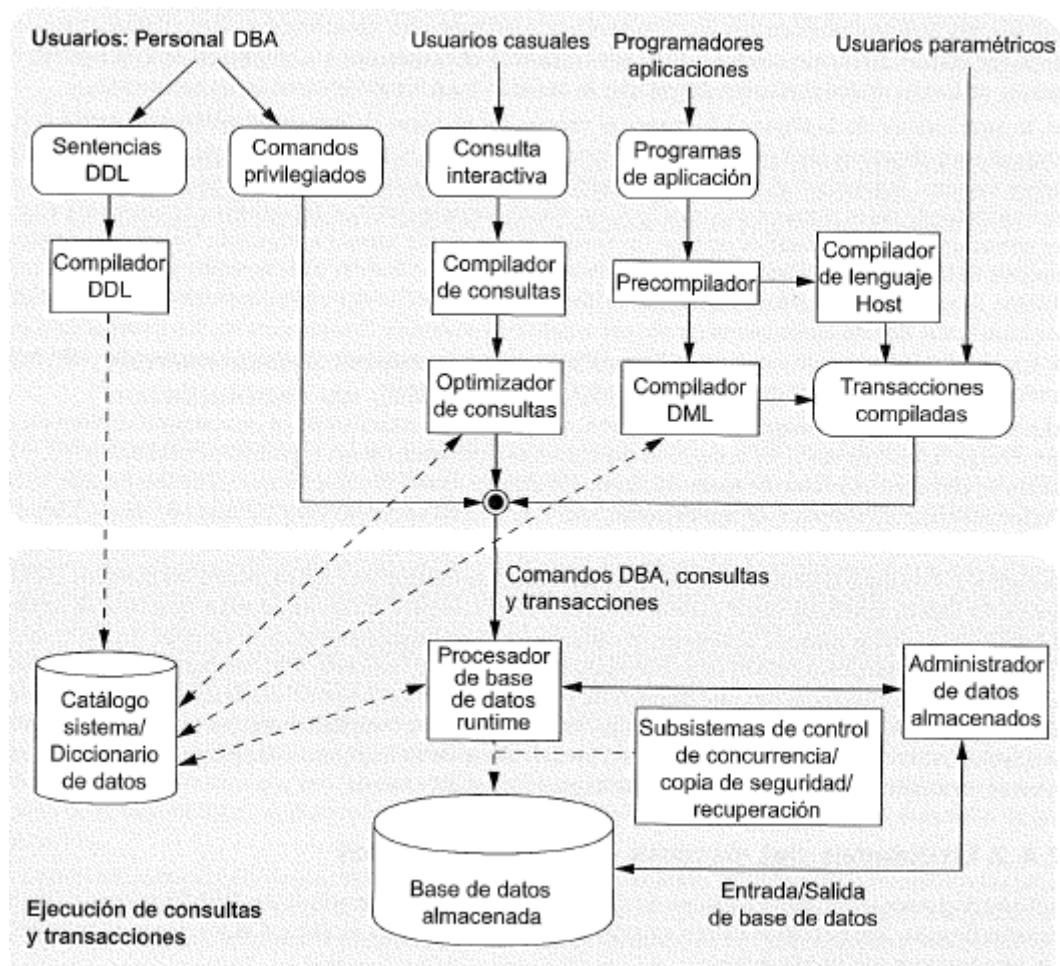
usuarios en particular está interesado y le oculta el resto de la base de datos.(también se basa en un modelo de datos de alto nivel)

(Está en las PPT, lo pongo como extra)

La arquitectura de tres niveles **ANSI** ocupa un lugar importante en el desarrollo de tecnologías de bases de datos porque separa el nivel externo de los usuarios, el nivel conceptual del sistema y el nivel de almacenamiento interno para diseñar una base de datos.

COMPONENTES DE UN DBMS

La base de datos y el catálogo del DBMS normalmente se almacenan en el disco. El acceso al disco está principalmente controlado por el Sistema Operativo, que planifica la entrada/salida del disco. Un módulo administrador de datos almacenados de alto nivel del DBMS controla el acceso a la información del DBMS almacenada en el disco, sea parte de la base de datos o del catálogo.



COMPILADOR DDL: procesa las definiciones de esquema, especificadas en el DDL, y almacena las descripciones de los esquemas (metadatos) en el catálogo del DBMS.

COMPILADOR DE CONSULTAS: analiza las consultas sintácticamente para garantizar la corrección de las operaciones de los modelos, los nombres de los elementos de datos, etcétera, y luego lo compila todo en un formato interno que está sujeto a optimización.

OPTIMIZADOR DE CONSULTAS: se ocupa de la reconfiguración y la posible reordenación de operaciones, eliminación de redundancias y uso de los algoritmos e índices correctos durante la ejecución. Consulta el catálogo del sistema para información estadística y física acerca de los datos almacenados, y genera un código ejecutable que lleva a cabo las operaciones necesarias para la consulta y realiza las llamadas al procesador runtime.

PRECOMPILADOR: extrae los comandos DML de un programa de aplicación escrito en un lenguaje de programación host. Estos comandos se envían al compilador DML. El resto del programa se envía al compilador de lenguaje host.

COMPILADOR DML: compila en código objeto y así accede a la base de datos. Los códigos objeto para los comandos DML y el resto del programa se enlazan, formando una transacción enlatada cuyo código ejecutable incluye llamadas al procesador de base de datos runtime.

PROCESADOR DE BASES DE DATOS RUNTIME: ejecuta los comandos privilegiados, los proyectos de consultas ejecutables y las transacciones enlatadas con parámetros runtime. Trabaja con el diccionario del sistema y se puede actualizar con estadísticas. Funciona con el administrador de datos almacenados, que a su vez utiliza servicios básicos del sistema operativo para ejecutar operaciones de entrada/salida de bajo nivel entre el disco y la memoria principal. Además se encarga de la administración de los búferes en la memoria principal.

SUBSISTEMA DE CONTROL DE CONCURRENCIA: subfunción del procesador de base de datos para colaborar con control de concurrencia.

ADMINISTRACIÓN DE DATOS ALMACENADOS: funciona con el procesador de base de datos para poder comunicarse con el sistema operativo y realizar entrada/salida.

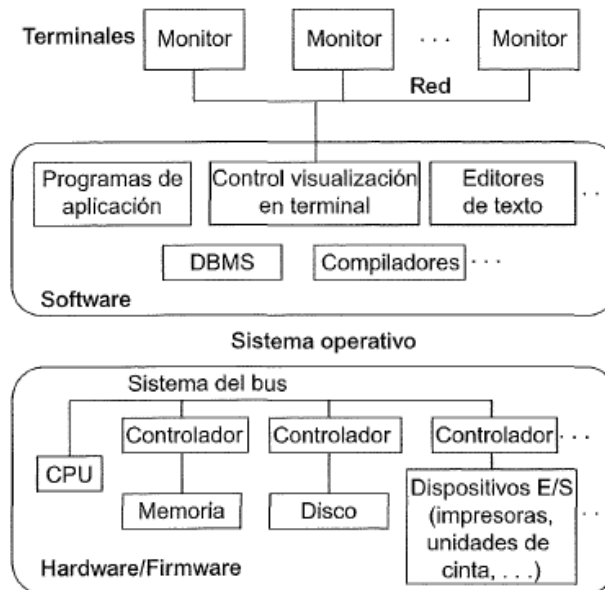
ARQUITECTURAS A IMPLEMENTAR PARA LOS DBMS

ARQUITECTURA CENTRALIZADA

Las arquitecturas primigenias utilizaban mainframes para proporcionar el procesamiento principal a todas las funciones del sistema, incluyendo las aplicaciones de usuario y los programas de interfaz de usuario, así como la funcionalidad del DBMS. La razón era que la mayoría de los usuarios accedía a esos sistemas a través de terminales de computador que no tenían potencia de procesamiento y solo ofrecían capacidades de visualización. Por lo tanto, todo el procesamiento se realizaba remotamente desde el computador a los terminales de visualización, que estaban conectados con el computador central a través de diferentes tipos de redes de comunicaciones.

Luego, la mayoría de los usuarios reemplazaron sus terminales por PCs y estaciones de trabajo aunque al principio se utilizaban de un modo muy parecido a las terminales de

visualización por que el DBMS seguía siendo centralizado, es decir, la ejecución de aplicaciones e interacción con el usuario se llevaba a cabo en una máquina.



ARQUITECTURA CLIENTE/SERVIDOR BÁSICA (SERVIDORES ESPECIALIZADOS)

Esta arquitectura se desarrolló para ocuparse de los entornos de computación en los que una gran cantidad de PCs, estaciones de trabajo, servidores de archivos, impresoras, servidores de bases de datos, servidores web y otros equipos están conectados a través de una red. La idea es definir servidores especializados con funcionalidades específicas. Se pueden definir servidores de impresión, servidores web o de e-mail. De este modo, muchas máquinas cliente pueden acceder a los recursos proporcionados por servidores especializados. Las máquinas cliente proporcionan al usuario las interfaces apropiadas para utilizar estos servidores, así como potencia de procesamiento local para ejecutar aplicaciones locales.

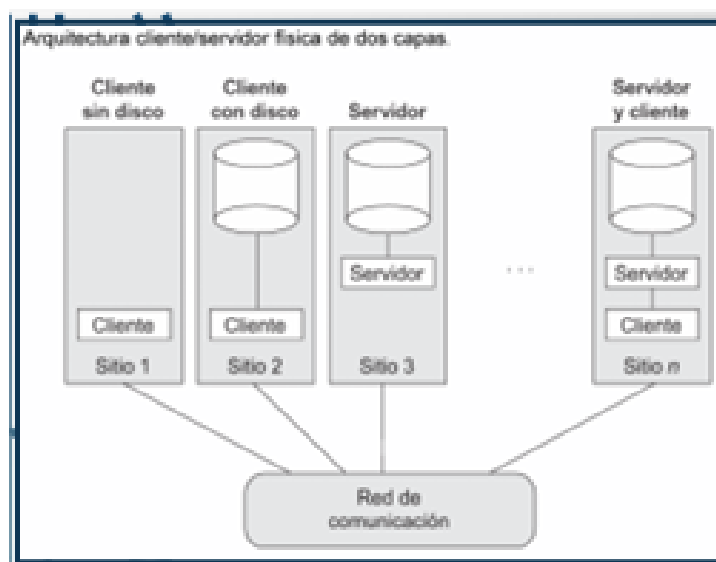


ARQUITECTURA CLIENTE/SERVIDOR DE DOS CAPAS

Algunos DBMS orientados a objetos adoptaron esta arquitectura en la cual los módulos software del DBMS se dividían entre cliente y servidor de un modo más integrado. El nivel servidor puede incluir la parte del software DBMS responsable de manipular los datos en las páginas del disco, controlar la concurrencia local y la recuperación, almacenar en buffer y en caché las páginas de disco, etc. Por otro lado, el nivel cliente puede manipular la interfaz de usuario, las funciones del diccionario de datos, las interacciones DBMS con los compiladores de lenguajes de programación, la optimización de consultas globales, el control de la concurrencia y la recuperación entre varios servidores, etc.

La interacción cliente/servidor es más estrecha y la realizan internamente los módulos DBMS, en lugar de los usuarios. El servidor se llama servidor de datos porque proporciona datos de las páginas de disco al cliente. El software DBMS del lado del cliente estructura después estos datos como objetos para los programas cliente.

Las ventajas de esta arquitectura son su simplicidad y su perfecta compatibilidad con los sistemas existentes.



ARQUITECTURA DE TRES CAPAS Y N CAPAS

Esta arquitectura añade una capa intermedia entre el cliente y el servidor de la base de datos.

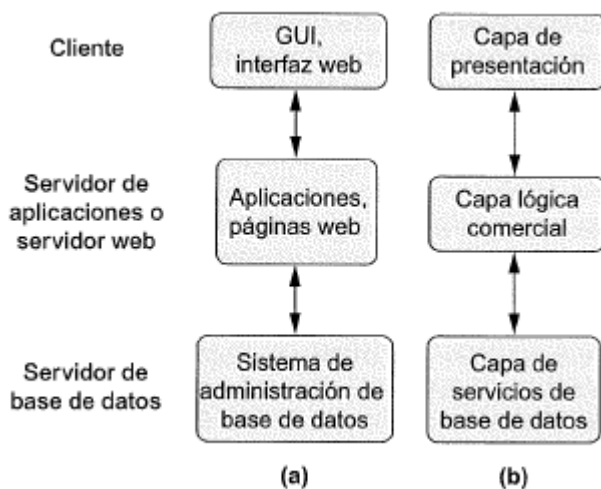
Esta capa intermedia se denomina servidor de aplicaciones o servidor web. Este juega un papel intermedio almacenando las reglas comerciales que se utilizan para acceder a los datos del servidor de bases de datos. Además, puede mejorar la seguridad de la base de datos al comprobar las credenciales del cliente antes de enviar una solicitud al servidor de la base de datos.

(Figura a) Los clientes contienen interfaces GUI y algunas reglas comerciales adicionales específicas de la aplicación. El servidor intermedio acepta las solicitudes del cliente, las procesa y envía comandos de bases de datos al servidor de bases de datos, y después actúa como un conducto para pasar datos procesados desde el servidor de bases de datos a los clientes, donde son procesados de forma más avanzada para su presentación en formato GUI a los usuarios.

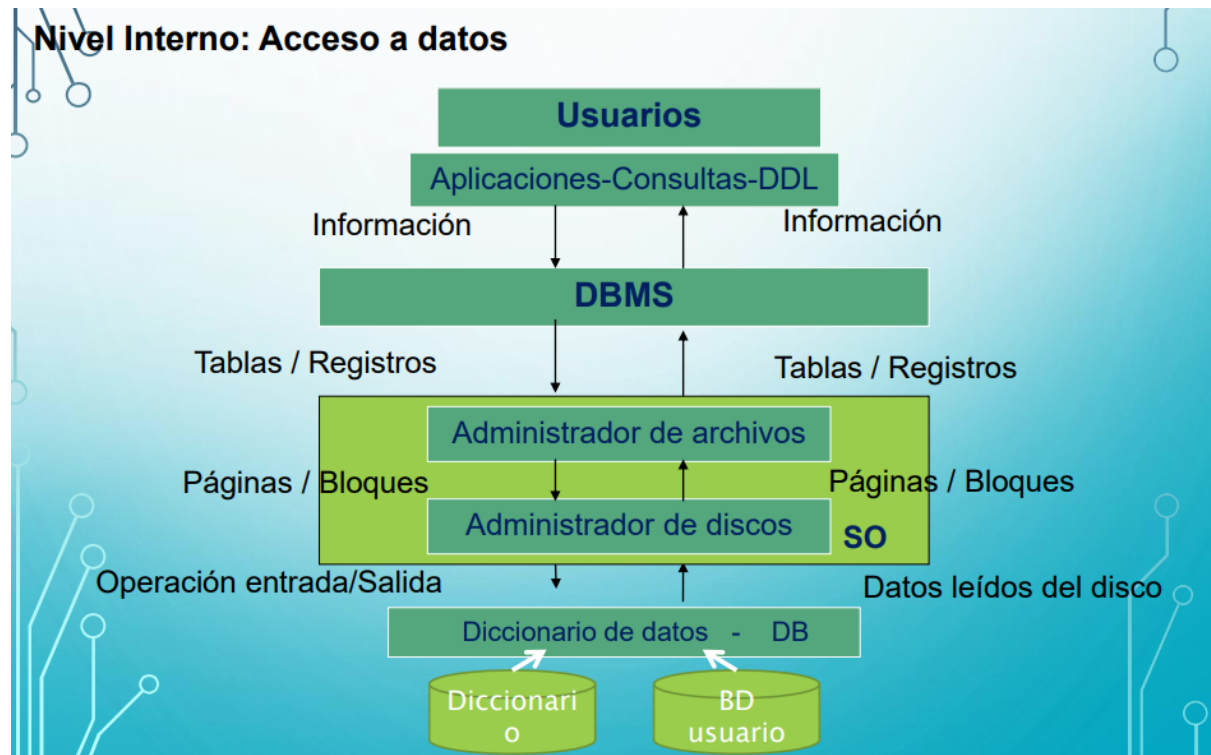
(Figura b) La capa de presentación muestra información al usuario y permite la entrada de datos. La capa lógica comercial manipula las reglas intermedias y las restricciones antes de que los datos sean pasados hacia arriba hasta el usuario, o hacia abajo, hasta el DBMS. La capa inferior incluye todos los servicios de administración de datos. Normalmente la lógica comercial está dividida en varias capas.

Es costumbre dividir las capas entre el usuario y los datos almacenados en componentes aún más sutiles para generar arquitecturas de n capas. Además de distribuir la programación y los datos por la red, las aplicaciones de n capas ofrecen la ventaja de que cualquiera de las capas se puede ejecutar en un procesador adecuado o plataforma de sistema operativo, además de poderse manipular independientemente.

Otra capa que se suele utilizar es la capa middleware, que da cuenta de los módulos front-end que comunican con una determinada cantidad de bases de datos back-end.



ACCESO A DATOS



ALMACENAMIENTO

ALMACENAMIENTO PRINCIPAL O PRIMARIO

Esta categoría abarca los medios de almacenamiento en los que la CPU puede operar, como la memoria principal(RAM) o la caché. Estos almacenamientos son de acceso rápido a los datos pero tienen una capacidad de almacenamiento limitada

ALMACENAMIENTOS SECUNDARIOS Y TERCARIOS

El almacenamiento secundario incluye los discos magnéticos, discos ópticos, cintas, y discos duros, mientras que los medios extraíbles se consideran como terciarios. Para operar con ellos, la CPU primero debe realizar una copia de su contenido en memoria principal.

“El proceso de diseño físico de la base de datos implica elegir las técnicas de organización de datos particulares que mejor se adapten a los requisitos de la aplicación. Esta etapa es importante para el diseñador de BD, para el DBA y para el implementador. Esta toma de decisiones puede influir en la performance”

CONCEPTOS DE ALMACENAMIENTO

- Bloque o página: tamaño establecido por el S.O al inicializar, es no dinámico. Es la unidad de transferencia entre disco y memoria
- Cluster: bloques contiguos es disco que pueden moverse a memoria como una unidad

- Dirección de bloque: para hacer una E/S se necesita nro de cilindro + nro de pista + nro de bloque
- Registros: colección de valores de datos que pueden interpretarse como hechos relacionados con las entidades, atributos y sus relaciones. Son los campos, se almacenan independientes con punteros.
- Ficheros: secuencia de registros - longitud fija y variable (varchar)
- Asignación de bloques: continua, enlazada, combinada, indexada
- Cabecera de fichero: para determinar longitud/orden de campos-separadores de registros



- Organización extendida y no extendida: cuando tenemos registros que no son divisores exactos del tamaño de los bloques, al llenar un bloque darán por resultado un espacio en los bloques de memoria que no pueden ser utilizados ya que no entraría un registro completo en dicho espacio. Esto también ocurre cuando el tamaño del registro es mayor al tamaño del bloque.

Para utilizar este espacio desaprovechado, podemos almacenar parte de un registro en un bloque y el resto en otro. Un puntero al final del primer bloque apunta al bloque que contiene el resto del registro, en caso de que no sean bloques consecutivos en el disco. Esta organización se denomina extendida porque los registros pueden abarcar más de un bloque. Siempre que un registro es más grande que un bloque, debemos utilizar una organización extendida. Si los registros no tienen permitido sobrepasar los límites de un bloque, se dice que la organización no es extendida.

En el caso de los registros de longitud variable, se puede utilizar cualquiera de las dos organizaciones, extendida o no extendida

OPERACIONES SOBRE FICHEROS

OPERACIONES DE RECUPERACIÓN: No cambian ningún dato del fichero, ya que únicamente localizan ciertos registros para que los valores de sus campos se puedan examinar o procesar.

OPERACIONES DE ACTUALIZACIÓN: modifica el fichero mediante la inserción o la eliminación de registros, modificando los valores de los campos.

ORGANIZACIÓN DE FICHEROS

FICHEROS DE REGISTROS DESORDENADOS (HEAP-PILA-SECUENCIAL)

Es el tipo de organización más sencillo y básico, según el cual los registros se guardan en el fichero en el mismo orden en que se insertan. Se utiliza a menudo con rutas de acceso adicionales, como los índices secundarios. También se utiliza para recopilar y almacenar registros de datos para un uso futuro.

La inserción de un registro nuevo es muy eficaz. El último bloque de disco del fichero se copia en el buffer, se añade el registro nuevo y se reescribe el bloque de nuevo en el disco. El aspecto negativo es que la búsqueda de un registro utilizando cualquier condición implica una búsqueda lineal bloque a bloque, por todo el fichero.

Para eliminar un registro, un programa primero debe buscar su bloque, copiar el bloque en un buffer, eliminar el registro del buffer y, por último, reescribir el bloque en el disco. Esto deja espacio inutilizado en el bloque del disco. La eliminación de una gran cantidad de registros de este modo supone un derroche de espacio de almacenamiento. Otra técnica que se utiliza para borrar registros es contar con un byte o un bit extra, denominado marcador de borrado, en cada registro. Un registro se borra estableciendo el marcador de borrado a un determinado valor. Un valor diferente del marcador indica que un registro no ha sido borrado. Los programas de búsqueda sólo consideran los registros válidos cuando realizan sus búsquedas. Estas dos técnicas de borrado requieren una reorganización periódica del fichero para reclamar el espacio inutilizado correspondiente a los registros borrados.

FICHEROS DE REGISTROS ORDENADOS-ORDENACIÓN FÍSICA

Los registros de un fichero se pueden ordenar fácilmente en el disco en función de los valores de uno de sus campos, denominado campo de ordenación. Esto conduce a un fichero ordenado o secuencial. Si el campo de ordenación también es un campo clave del fichero, entonces el campo se denomina clave de ordenación.

Los registros ordenados tienen algunas ventajas sobre los ficheros desordenados. En primer lugar, la lectura de los registros en el orden marcado por los valores de la clave de ordenación es extremadamente eficaz porque no se necesita una ordenación. En segundo lugar, encontrar el siguiente registro al actual según el orden de la clave de

ordenación normalmente no requiere acceder a bloques adicionales porque el siguiente registro se encuentra en el mismo bloque que el actual. En tercer lugar, el uso de una condición de búsqueda basándose en el valor de un campo clave de ordenación ofrece un acceso más rápido cuando se utiliza la técnica de búsqueda binaria, que constituye una mejora respecto a las búsquedas lineales.

La ordenación no ofrece ninguna ventaja para el acceso aleatorio u ordenados de los registros basándose en los valores de otros campos no ordenados del fichero. Para acceder a los registros en orden basándose en un campo desordenado, es necesario crear otra copia ordenada del fichero.

La inserción y eliminación de registros son operaciones costosas para un archivo ordenado, porque los registros deben permanecer físicamente ordenados. Para insertar un registro debemos entrar su posición correcta en el fichero basándonos en el valor de su campo de ordenación, y después hacer espacio en el fichero para insertar el registro en esa posición. En la eliminación de un registro, el problema es menos grave si se utilizan marcadores de borrado y una reorganización periódica.

Un método para que la inserción sea más eficaz es crear un fichero temporal desordenado denominado fichero de desbordamiento o transacciones. Con esta técnica, el fichero ordenado actual se denomina fichero principal o maestro. Los registros nuevos se insertan al final del fichero de desbordamiento, en lugar de hacerlo en su posición correcta en el fichero principal. Periódicamente, el fichero de desbordamiento se ordena y mezcla con el maestro durante la reorganización del fichero. La inserción es muy eficaz, pero a costa de aumentar la complejidad del algoritmo de búsqueda.

TÉCNICAS DE DISPERSIÓN (HASH)

Organización de ficheros que proporciona un acceso muy rápido a los registros bajo ciertas condiciones de búsqueda. Esta organización se denomina normalmente fichero disperso o hash. La condición de búsqueda debe ser una condición de igualdad sobre un campo, denominado campo de dispersión o campo hash. En la mayoría de los casos, el campo de dispersión también es un campo clave del fichero, en cuyo caso se denomina clave de dispersión. La idea que hay detrás de la dispersión es proporcionar una función h , denominada función de dispersión o función de aleatorización, que se aplica al valor del campo de dispersión de un registro y produce la dirección del bloque de disco en el que está almacenado el registro.

En los ficheros internos, la dispersión normalmente se implementa como una tabla de dispersión mediante el uso de un array de registros.

Vamos a suponer que el índice del array va de 0 a $M-1$, por lo tanto tenemos M slots cuyas direcciones corresponden a los índices del array. Una función de dispersión común es $h(K) = K \bmod M$, que devuelve el resto del valor entero de un campo de dispersión después de dividirlo por M ; este valor se utiliza después para la dirección del registro.

Una colisión se produce cuando el valor del campo de dispersión de un registro que se está insertando se dispersa a una dirección que ya contiene un registro diferente. En esta situación, debemos insertar el registro nuevo en alguna otra posición, puesto que su

dirección de dispersión está ocupada. El proceso de encontrar otra posición se denomina resolución de colisiones. Hay varios métodos para resolver una colisión:

- **DIRECCIONAMIENTO ABIERTO:** a partir de la posición ocupada especificada por la dirección de dispersión, el programa comprueba las posiciones subsiguientes en orden hasta encontrar una posición sin utilizar.
- **ENCADENAMIENTO:** para este método, se conservan varias ubicaciones de dispersión, normalmente extendiendo el array con algunas posiciones de desbordamiento. Adicionalmente, se añade un campo puntero a cada ubicación de registro. Una colisión se resuelve colocando el registro nuevo en una ubicación de desbordamiento sin utilizar y estableciendo el puntero de la ubicación de la dirección de dispersión ocupada a la dirección de esa ubicación de desbordamiento. Se conserva entonces una lista enlazada de registros de desbordamiento por cada dirección de dispersión.
- **DISPERSIÓN MÚLTIPLE:** el programa aplica una segunda función de dispersión si la primera desemboca en una colisión. Si se produce otra colisión, el programa utiliza el desbordamiento abierto o aplica una tercera función de dispersión y utiliza después el direccionamiento abierto si es necesario.

El objetivo de una buena función de dispersión es distribuir uniformemente los registros sobre el espacio de direcciones para minimizar las colisiones sin dejar demasiadas ubicaciones sin utilizar.

INDEXACIÓN

Para un fichero con una estructura de registro dada compuesta por varios campos, normalmente se define una estructura de índice con un solo campo del fichero, que se conoce como campo de indexación. Normalmente, el índice almacena todos los valores del campo de índice, junto con una lista de los punteros a todos los bloques de disco que contienen los registros con ese valor de campo.

- El fichero de índice es mucho más pequeño lo que permite acelerar el acceso a datos.
- Se puede utilizar cualquier campo para indexar, incluso campos compuestos
- Un fichero puede tener varios índices de datos
- Los índices ofrecen rutas de acceso que no afectan a la organización física del fichero
- Se deben considerar la actualización de los índices generados
- Actualmente los DBMS generan índices por PK y FK

CLASIFICACIÓN DE LOS ÍNDICES

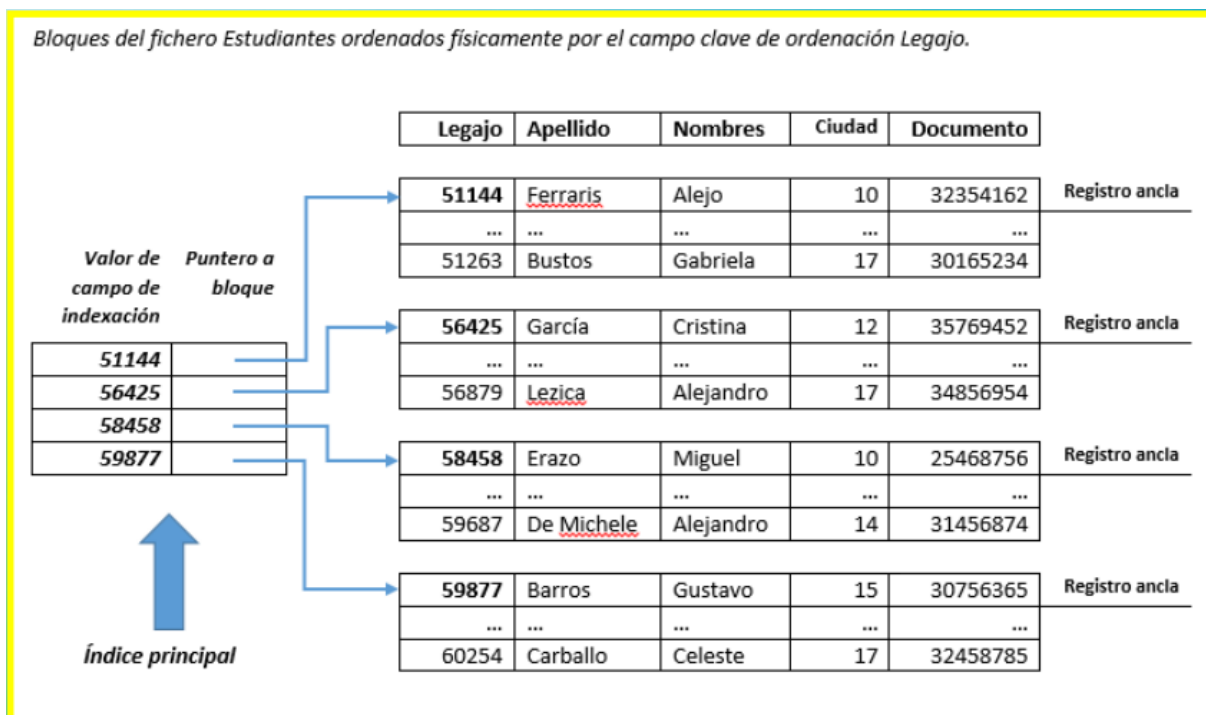
“Los índices se pueden clasificar como densos o escasos. Un índice denso tiene una entrada de índice por cada valor de la clave de búsqueda (y, por tanto, cada registro) del fichero de datos. Un índice escaso (o no denso) sólo tiene entradas para algunos de los valores de búsqueda.”

TIPOS DE ÍNDICES ORDENADOS DE UN NIVEL

1. Principal:

Un índice principal es un fichero ordenado cuyos registros son de longitud fija con dos campos. El primer campo es del mismo tipo de datos que el campo clave de ordenamiento (clave principal) del fichero de datos, y el segundo campo es un puntero a un bloque de disco. En el fichero índice hay una entrada de índice por cada bloque del fichero de datos. Cada entrada del índice tiene dos valores: el valor del campo principal para el primer registro de un bloque, y un puntero a ese bloque. El primer registro de cada bloque del fichero de datos es el registro ancla del bloque.

- Precondición: El fichero de datos debe estar ordenado físicamente por campo clave
- No contiene valores repetidos
- Es un índice escaso (no denso)
- Ocupa menos espacio que el fichero de datos
- Entran más registros del fichero índice en cada bloque
- Se realiza búsqueda binaria en el fichero índice y lectura del bloque
- Actualización:
 - Inserción es un problema: pueden cambiar los registros anclas o usar ficheros de desbordamiento
 - Borrado: se usa borrado con marcador

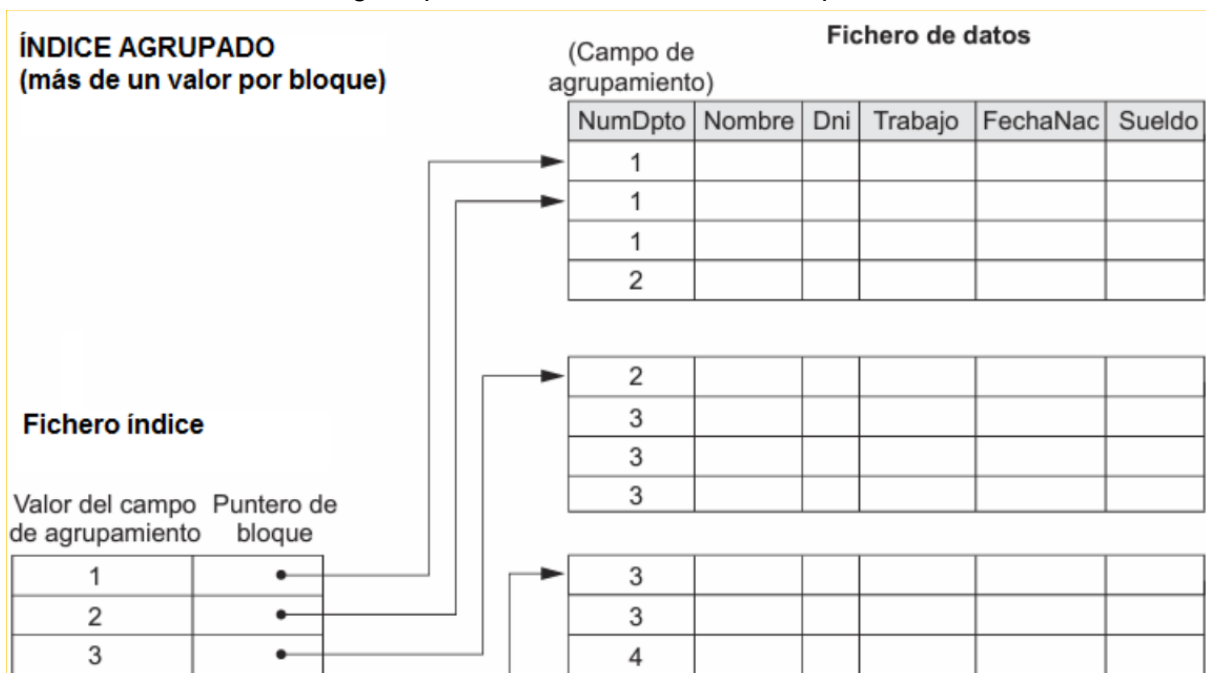


2. Agrupado:

Si los registros del fichero están ordenados físicamente por un campo no clave, este campo se denomina campo agrupado. Podemos crear un tipo de índice que permite acelerar la recuperación de los registros que tienen el mismo valor para el campo agrupado.

Un índice agrupado es un fichero ordenado con dos campos: el primero es del mismo tipo que el campo agrupado del fichero de datos, y el segundo es un puntero a un bloque. En el índice agrupado hay una entrada por cada valor distinto del campo agrupado, que contiene el valor y un puntero al primer bloque del fichero de datos que tiene un registro con ese valor para su campo agrupado.

- Precondición: Fichero de datos ordenados físicamente por campo no clave
- Es un índice escaso (no denso)
- Ocupa menos espacio que el fichero de datos, registro de longitud fija
- Entran más registros del fichero índice en cada bloque
- Búsqueda binaria en el fichero índice y lectura del bloque
- Actualización:
 - Inserción y borrado son un problema: reservar un bloque o bloques contiguos por cada valor distinto del campo de indexación.



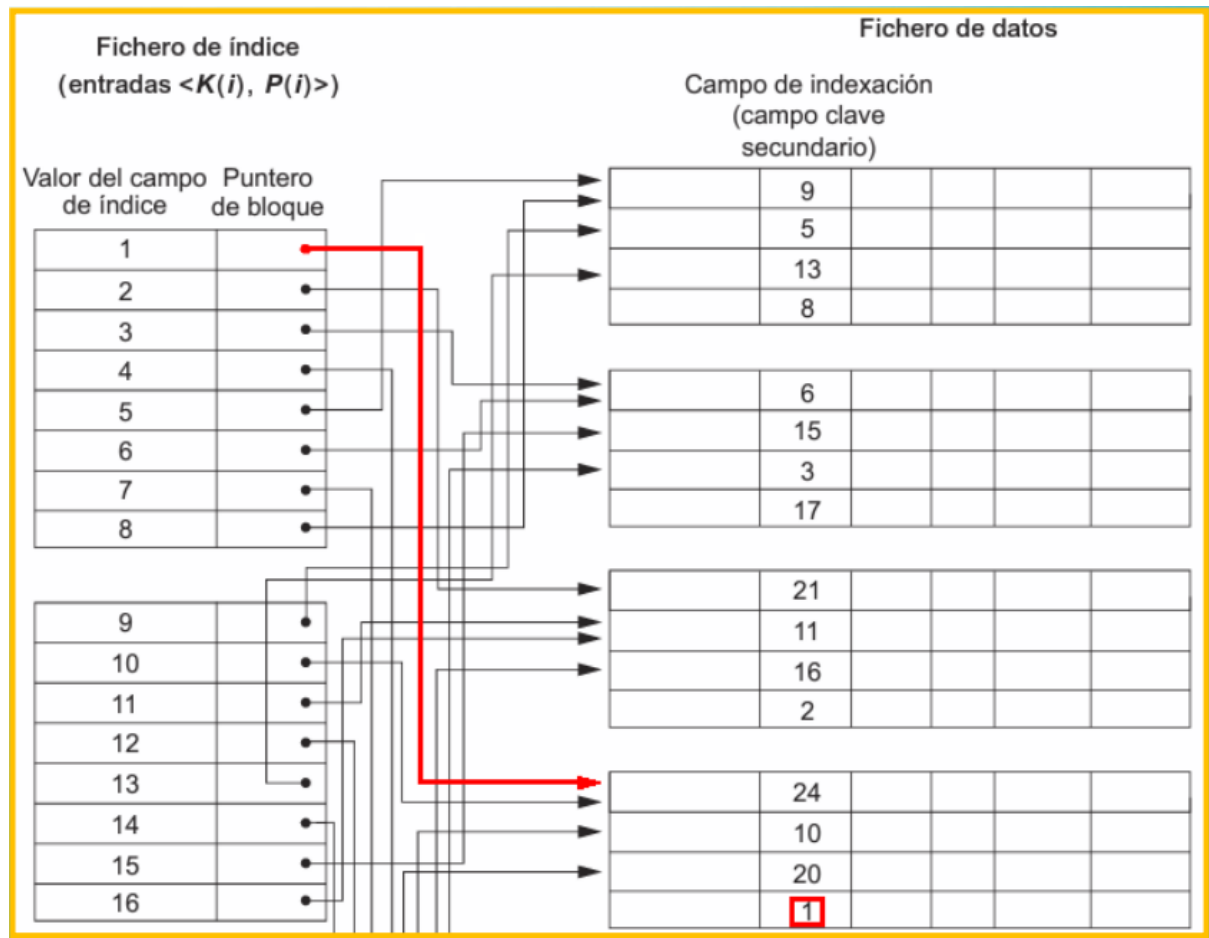
3. Secundario:

Un índice secundario proporciona un medio secundario de acceso a un fichero para el que ya existe algún acceso principal. El Índice secundario puede ser un campo que es una clave candidata y tiene un valor único en cada registro, o puede ser una "no clave" con valores duplicados. El Índice es un fichero ordenado con dos campos. El primero es del mismo tipo de datos que algún campo no ordenado del fichero de datos que es un campo de indexación. El segundo campo es un puntero de bloque o un puntero de registro. Puede haber muchos Índices secundarios (y, por tanto, campos de indexación) para el mismo fichero.

Un Índice secundario normalmente necesita más espacio de almacenamiento y un tiempo de búsqueda mayor que un Índice principal, debido a su mayor cantidad de entradas. Sin embargo, la mejora en el tiempo de búsqueda de un registro arbitrario es mucho mayor para un Índice secundario que para un Índice principal.

- Precondición: el fichero ya tiene un índice
- Puede haber muchos índices secundarios por fichero de datos
- Proporciona un orden lógico a los registros almacenados

- denso o no denso

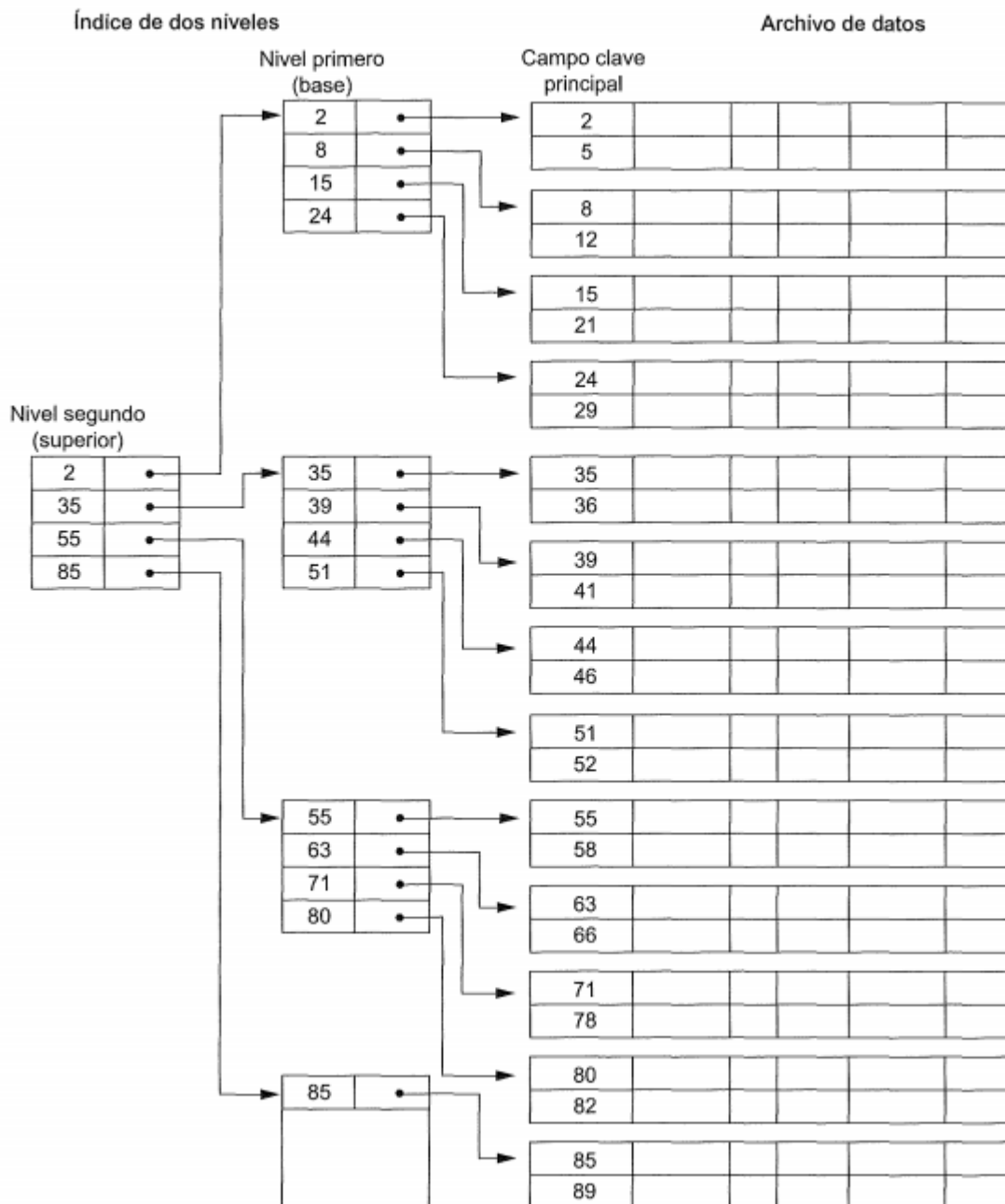


ÍNDICES MULTINIVEL

Se aplican después de generar un índice ya sea principal, agrupado o secundario.

Un índice multinivel considera el fichero de índice, llamado primer nivel o base de un índice multinivel, como fichero ordenado con un valor distinto por cada entrada. Luego, podemos crear un índice principal para el primer nivel. Este índice al primer nivel se denomina segundo nivel del índice multinivel. Como el segundo nivel es un índice principal, podemos utilizar anclas de bloque de modo que el segundo nivel tenga una sola entrada por cada bloque del primer nivel.

Podemos repetir este proceso para el segundo nivel creando así el tercer nivel. Este proceso se repite hasta que todas las entradas entren en un solo bloque, a este nivel se lo conoce como nivel de índice superior.



BASES DE DATOS DISTRIBUIDAS (BDD)

Podemos definir una DDB (Base de datos distribuida, Distributed DataBase) como una colección de múltiples bases de datos **distribuidas** interrelacionadas de forma lógica sobre una red de computadores, y un DDBMS (Sistema de administración de bases de datos distribuidas, Distributed DataBase Management System) como el software encargado de administrar la base de datos distribuida mientras hace la distribución transparente para el usuario.

Un sistema de computación distribuido consiste en un número de elementos de procesamiento, no necesariamente homogéneos, que están interconectados mediante una red de computadores, y que cooperan para la realización de ciertas tareas asignadas. Estos

sistemas dividen un gran e inmanejable problema en piezas más pequeñas para resolverlo de una manera coordinada.

VENTAJAS DE LAS BDD

1. **Administración de datos distribuidos con distintos niveles de transparencia:**

De manera ideal, un DBMS debe ser una distribución transparente en el sentido de ocultar los detalles de dónde está físicamente ubicado cada fichero dentro del sistema.

- Transparencia de red o de distribución: Hace referencia a la autonomía del usuario de los detalles operacionales de la red. Puede dividirse en:
 - transparencia de localización: hace mención al hecho de que el comando usado para llevar a cabo una tarea es independiente de la ubicación de los datos y del sistema desde el que se ejecutó dicho comando.
 - transparencia de denominación: implica que, una vez especificado un nombre, puede accederse a los objetos nombrados sin ambigüedad y sin necesidad de ninguna especificación adicional.
- Transparencia de replicación: Pueden almacenarse copias de los datos en distintos lugares para disponer de una mayor disponibilidad, rendimiento y fiabilidad. Permite que el usuario no se entere de la existencia de copias.
- Transparencia de fragmentación: Existen dos posibles tipos de fragmentación la horizontal y la vertical. Una consulta global del usuario debe ser transformada en varias consultas fragmentadas. La transparencia de fragmentación permite que el usuario no se entere de la existencia de fragmentos.
- La transparencia de diseño y de ejecución: hace referencia a la libertad de saber cómo está diseñada la base de datos distribuida y dónde ejecuta una transacción.

2. **Incremento de la fiabilidad y la disponibilidad:** La fiabilidad está definida ampliamente como la probabilidad de que un sistema esté funcionando (no caído) en un momento de tiempo, mientras que la disponibilidad es la probabilidad de que el sistema está continuamente disponible durante un intervalo de tiempo. Cuando los datos y el software DBMS están distribuidos a lo largo de distintas localizaciones, uno de ellos puede fallar, mientras el resto continúa operativo. Sólo los datos y el software almacenados en la localización que falla serán los que no estén disponibles. Esto mejora tanto la fiabilidad como la disponibilidad. Se logra una apreciable mejora al replicar tanto los datos como el software en más de una ubicación.

3. **Rendimiento mejorado:** Un DBMS distribuido fragmenta la base de datos manteniendo la información lo más cerca posible del punto donde es más necesaria. La localización de datos reduce el enfrentamiento por la CPU y los servicios de E/S, a la vez que atenúa los retardos en el acceso implícito a las redes de área

extendida. Cuando distribuimos una base de datos obtenemos bases de datos más pequeñas, como resultado, las consultas locales y las transacciones de acceso a los datos de uno de estos sitios tienen un mayor rendimiento debido al menor tamaño de esas bases de datos.

4. **Expansión más sencilla:** La expansión del sistema en términos de incorporación de más datos, incremento del tamaño de las bases de datos o la adición de más procesadores es mucho más sencilla.

FUNCIONES ADICIONALES DE LA DDBMS

1. **Seguimiento de los datos.** La capacidad de controlar la distribución de los datos, la fragmentación y la replicación expandiendo el catálogo DDBMS.
2. **Procesamiento de consultas distribuidas.** La posibilidad de acceder a sitios remotos y de transmitir consultas y datos a lo largo de todos esos sitios mediante una red de comunicación.
3. **Administración de transacciones distribuidas.** La facultad de diseñar estrategias de ejecución de consultas y transacciones que accedan a los datos desde más de una ubicación y de sincronizar el acceso a los datos distribuidos y de mantener la integridad de toda la base de datos.
4. **Administración de datos replicados.** La capacidad de decidir a qué copia de un dato acceder y de mantener la consistencia de las copias de un elemento de datos replicado.
5. **Recuperación de una base de datos distribuida.** La facultad de recuperarse de las caídas de una localización individual u otro tipo de fallos
6. **Seguridad.** Las transacciones distribuidas deben ejecutarse con una adecuada administración de la seguridad de los datos y contando con los privilegios de autorización/acceso de los usuarios.
7. **Administración del directorio (catálogo) distribuido.**

“Estas funciones en sí mismas aumentan la complejidad de un DDBMS en relación a un DBMS centralizado.”

TÉCNICAS DE DISEÑO DE BDD

1. Fragmentación: podemos partir una base de datos en fragmentos los cuales son unidades lógicas que se pueden almacenar en varios sitios:
 - Fragmentación Horizontal: La fragmentación horizontal divide una relación horizontalmente agrupando filas para crear subconjuntos de tuplas, cada uno de ellos con un cierto significado lógico. Las tuplas pertenecientes al fragmento horizontal se especifican mediante una condición sobre uno o más atributos de la relación.
 - Fragmentación Vertical: La fragmentación vertical divide una relación "verticalmente" por columnas. Un fragmento vertical de una relación sólo mantiene ciertos atributos de la misma. Es necesario incluir la clave primaria, o alguna otra clave candidata, en cada fragmento vertical de modo que nos permita reconstruir la relación completa a partir de ellos.

- Fragmentación mixta: Podemos entremezclar los dos tipos de fragmentación para obtener una fragmentación mixta.
2. Replicación: Si un fragmento se encuentra en más de un sitio se dice que está replicado. La replicación es útil para mejorar la disponibilidad de los datos.
- Total: El caso más extremo es la replicación de toda la base de datos en cada sitio del sistema distribuido. Esto puede mejorar considerablemente la disponibilidad, ya que el sistema puede seguir funcionando con tal de que uno de los sitios esté activo. También mejora el rendimiento en la recuperación de consultas globales porque los resultados de este tipo de consultas pueden obtenerse localmente desde cada uno de estos sitios siempre y cuando incluya un módulo del servidor. Desventaja: las operaciones de actualización de datos se vuelven mucho más lentas y las técnicas de control de concurrencia y recuperación se vuelven más caras.
 - Parcial: algunos fragmentos de la base de datos pueden replicarse mientras que otros no.

“Cada fragmento, o cada copia del mismo, debe estar asignado a un sitio concreto del sistema distribuido. Este proceso recibe el nombre de **distribución de datos** (o **ubicación de datos**). La elección del sitio y el grado de replicación dependen de los objetivos de rendimiento y la disponibilidad del sistema.”

DIFERENCIAS A NIVEL HARDWARE

- Existen múltiples computadores llamados sitios o nodos.
- Estos sitios deben estar conectados por algún tipo de red de comunicación para transmitir los datos y los comandos entre ellos

Estos sitios pueden estar cercanos entre sí y conectados mediante una red de **área local**, o estar geográficamente distribuidos a larga distancia y enlazados a través de una red de **área expandida o long-haul**.

Las redes pueden tener diferentes topologías que definen las rutas de comunicación directas entre los sitios. Estas pueden tener un efecto significativo sobre el rendimiento y, por ello, sobre las estrategias de distribución del procesamiento de las consultas y el diseño de la base de datos distribuida.

Tipos de Sistemas de DDB:

- Grado de Homogeneidad del DDBMS: Homogéneo / Heterogéneo: Un sistema DDB será homogéneo si todos sus nodos utilizan el mismo motor de bases de datos.
- Con o sin Autonomía Local: Si cada nodo puede resolver consultas por sí solo sin necesidad de consultar a otros nodos se dice que tiene autonomía local.

CUESTIONES QUE CAMBIAN ENTRE UNA BD CENTRALIZADA Y UNA BDD

- **Costo de transferencia de datos.** Mayor en las bases distribuidas.

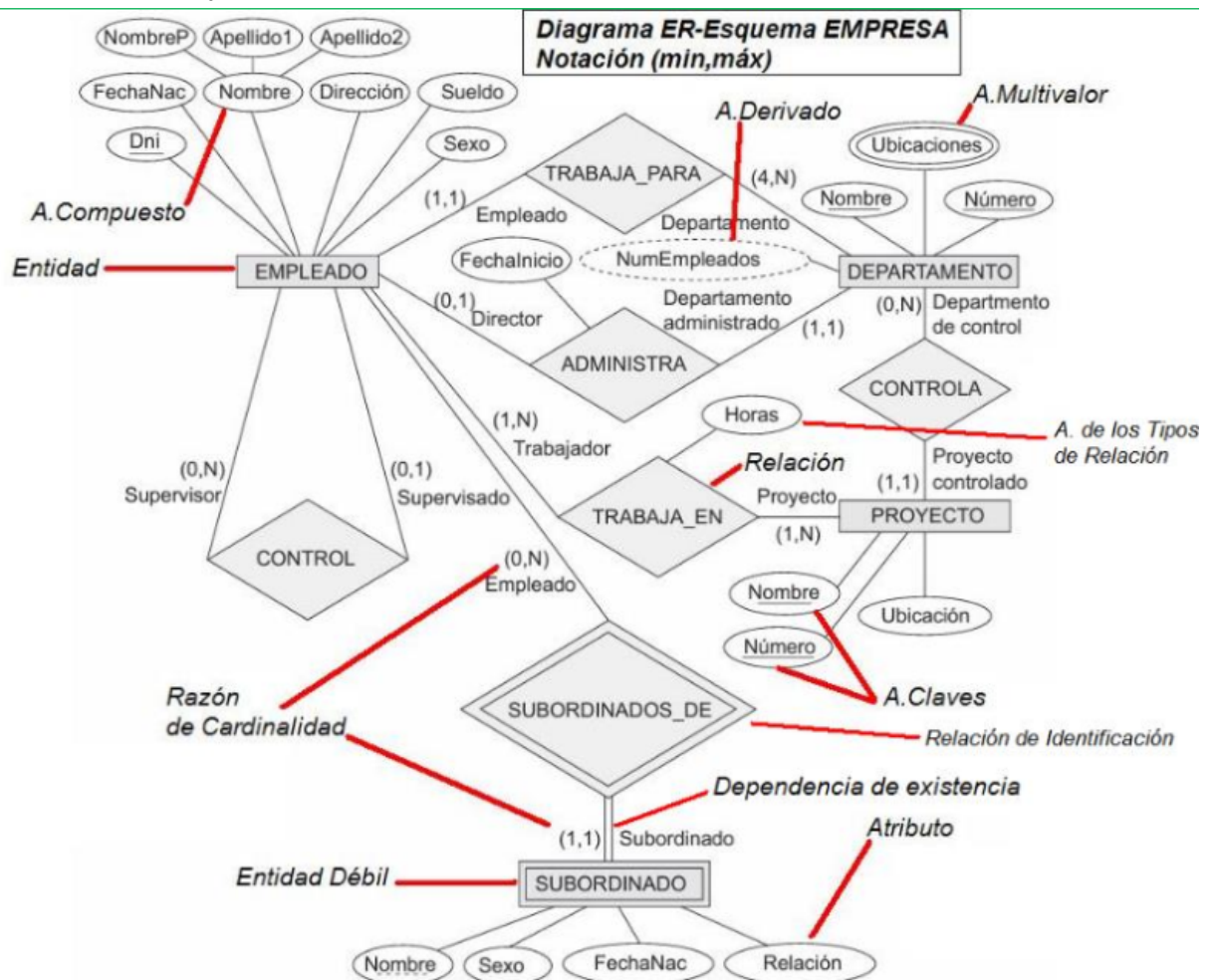
- **Tratar con múltiples copias de los datos.** El método de control de la concurrencia es responsable de mantener la consistencia de todas esas copias.
- **Fallo de los sitios individuales.** En caso de ser posible, el DBMS debe seguir operando con los sitios que están en funcionamiento cuando uno o más de estos sitios fallan.
- **Fallo de los enlaces de comunicación.** El sistema debe ser capaz de tratar con los fallos que se produzcan en los enlaces de comunicación que conectan los sitios.
- **Confirmación distribuida.** Pueden aparecer problemas a la hora de confirmar (commit) una transacción que está accediendo a las bases de datos almacenadas en varios sitios si alguno de ellos falla durante el proceso.
- **Estancamiento distribuido.** El interbloqueo (deadlock) puede producirse entre varios sitios, por lo que deben extenderse las técnicas para gestionarlo

MODELOS DE DATOS

CLASIFICACIÓN

BASADOS EN OBJETOS

- Entidad-Relación: Es un modelo de datos conceptual de alto nivel. Este modelo y sus variaciones se utilizan con frecuencia para el diseño conceptual de las aplicaciones de base de datos.
A la hora de realizar un modelo de DB, el primer paso es la recopilación de requisitos y el análisis. Una vez hecho esto, el siguiente paso es crear un esquema conceptual para la base de datos, mediante un modelo de datos conceptual de alto nivel. El esquema conceptual es una descripción concisa de los requisitos de datos por parte de los usuarios e incluye descripciones detalladas de los datos expresados como entidades, relaciones y restricciones. Estos modelos no incluyen aspectos de implementación y son fáciles de entender para los clientes



Orientado a Objetos:

Surgen por necesidad de las aplicaciones:

- Estructura de datos complejas
- Tipos de datos
- Transacciones larga duración
- Operaciones específicas.

Incremento en el uso de del POO y la persistencia de los objetos.

- Hay prototipos y productos como Gemstone/s,
- Comité de ODBMS para estandarizar: ODMG
- El sistema genera un OID **immutable** para cada objeto
- A cada objeto se lo conoce como(i, c, v)
 - i: OID
 - c: constructor del tipo (atom, tuple, set, list, bag, array)
 - v: valor
- Encapsulamiento de operaciones: en la definición de objetos.
- Lenguajes: ODL, OQL

```

$$o_1 = (i_1, \text{atom}, \text{'Madrid'})$$

$$o_2 = (i_2, \text{atom}, \text{'Valencia'})$$

$$o_3 = (i_3, \text{atom}, \text{'Sevilla'})$$

$$o_4 = (i_4, \text{atom}, 5)$$

$$o_5 = (i_5, \text{atom}, \text{'Investigación'})$$

$$o_6 = (i_6, \text{atom}, \text{'22-05-1988'})$$

$$o_7 = (i_7, \text{set}, \{i_1, i_2, i_3\})$$

$$o_8 = (i_8, \text{tuple}, \langle \text{NombreDpto}:i_5, \text{NúmeroDpto}:i_4, \text{Dire}:i_9, \text{Ubicaciones}:i_7, \text{Empleados}:i_{10}, \text{Proyectos}:i_{11} \rangle)$$

$$o_9 = (i_9, \text{tuple}, \langle \text{Dire}:i_{12}, \text{FechaIngresoDirector}:i_6 \rangle)$$

$$o_{10} = (i_{10}, \text{set}, \{i_{12}, i_{13}, i_{14}\})$$

```

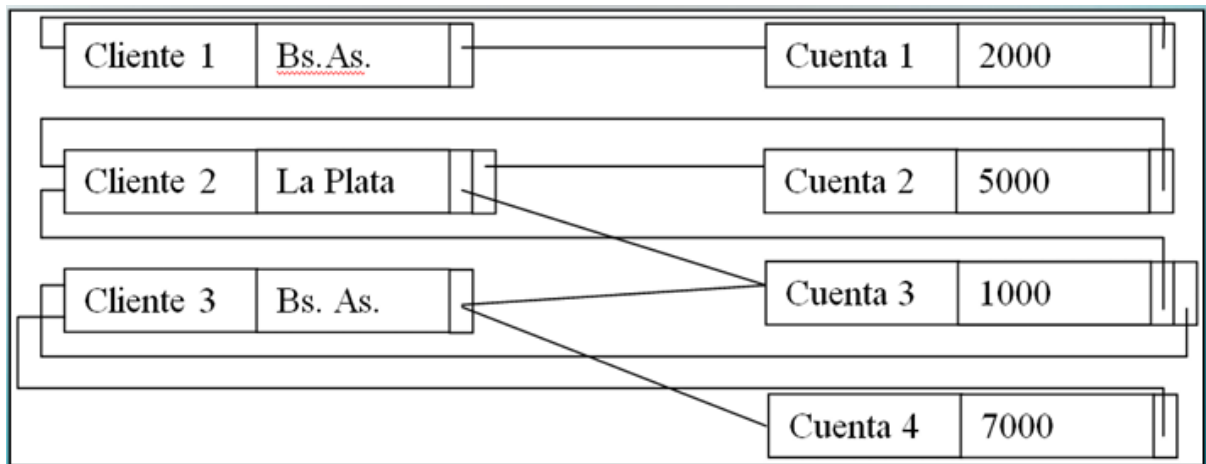
```
define class EMPLEADO
  type tuple ( Nombre:      string;
                Apellido1:  string;
                Apellido2:  string;
                Dni:         string;
                FechaNac:   FECHA;
                Dirección:  string;
                Sexo:       char;
                Sueldo:     float;
                Supervisor: EMPLEADO;
                Dept:       DEPARTAMENTO; );
  operations edad:         integer;
                crear_emp:  EMPLEADO;
                destruir_emp: boolean;
end EMPLEADO;
```

BASADOS EN REGISTROS

Red:

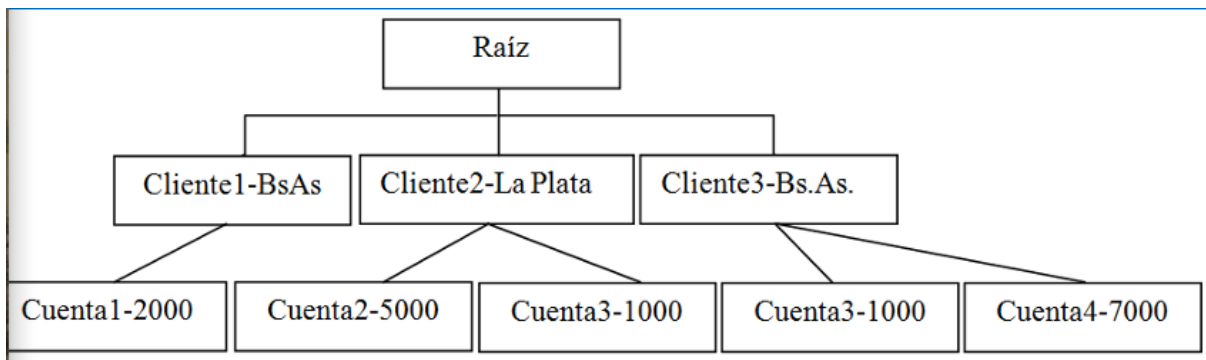
- Datos: representados como colecciones de registros -estructura arbitraria-

- Relaciones: Son punteros o enlaces



Jerárquico:

- Implementación particular del Modelo de Red
- Datos: representados como colección de registros pero estructura padre/hijo
- Relaciones: son punteros o enlaces
- Permite relaciones: Uno a uno y Uno a muchos.



Relacional:

El modelo relacional representa a la base de datos como una colección de relaciones. Cada una de estas relaciones se parece a una tabla de valores.

Cuando una relación está pensada como una tabla de valores, cada fila representa una colección de valores relacionados. El nombre de la tabla se utiliza para ayudar a interpretar el significado de cada uno de los valores de la fila. Los nombres de columna, especifican el modo de interpretar los valores de cada fila de la columna en la que se encuentren.

“En la terminología formal del modelo relacional, una fila recibe el nombre de tupla, una cabecera de columna es un atributo y el nombre de la tabla una relación. El tipo de dato que describe los valores que pueden aparecer en cada columna está representado por un dominio de posibles valores.”

- Tupla: colección de valores relacionados.
- Atributo: es cada columna de la cabecera
- Dominio: conjunto de valores atómicos que puede asumir un atributo
- Grado: número de atributos en la cabecera
- Cardinalidad: número de filas en la relación.

“Valor atómico es aquel que es indivisible en el dominio del modelo relacional.”

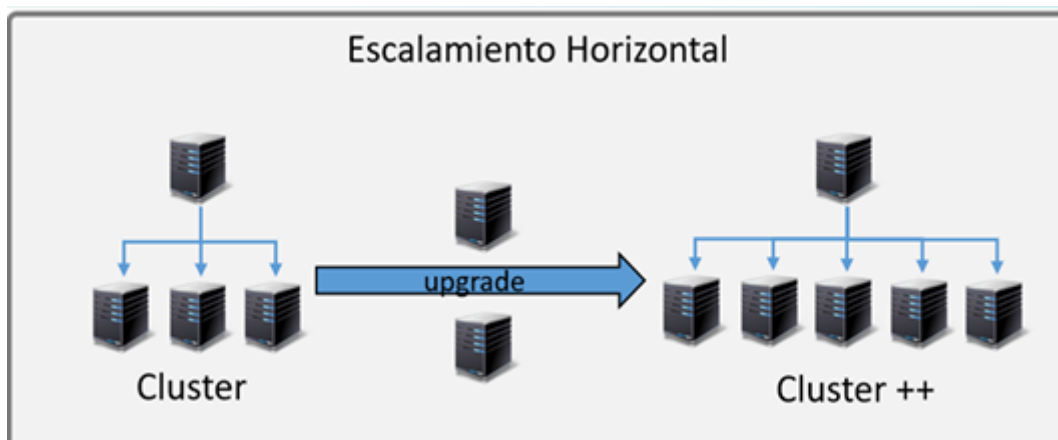
Las transacciones garantizan las propiedades ACID

- **Atomicidad.** Una transacción es una unidad atómica de procesamiento; o se ejecuta en su totalidad o no se ejecuta en absoluto.
- **Conservación de la consistencia.** Una transacción está conservando la consistencia si su ejecución completa lleva a la base de datos de un estado consistente a otro.
- **Aislamiento.** Una transacción debe aparecer como si estuviera ejecutándose de forma aislada a las demás. Es decir, la ejecución de una transacción no debe interferir con la ejecución de ninguna otra transacción simultánea.
- **Durabilidad.** Los cambios aplicados a la base de datos por una transacción confirmada deben persistir en la base de datos. Estos cambios no deben perderse por culpa de un fallo.

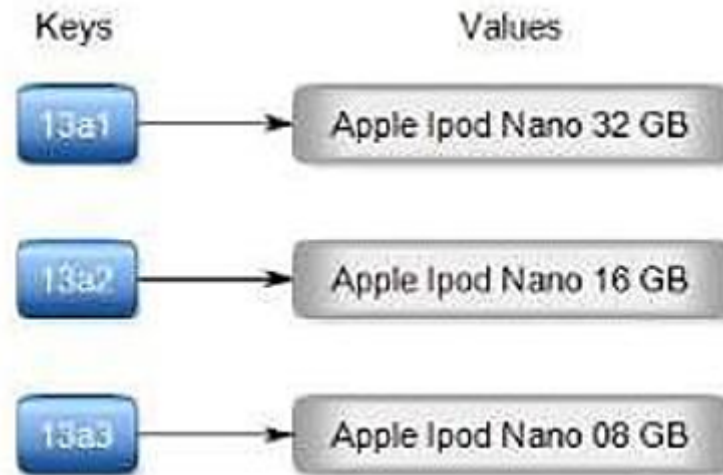
BD NO RELACIONALES (NO SQL- NOT ONLY SQL)

Características de NoSQL (Not Only SQL):

- No usan SQL como lenguaje de consulta estándar
- La información almacenada no requiere formato o esquema (schemaless)
- Permiten escalabilidad horizontal
- No garantizan ACID para mejorar rendimiento y disponibilidad
- Presentan operaciones atómicas
- Permiten replicación y distribución
- Almacenamiento de grandes bancos de información



Clave-Valor: Sencilla en cuanto a funcionalidad. Cada elemento está identificado por una llave única, lo que permite la recuperación de la información de forma muy rápida.



Documentales: Se pueden almacenar datos de tipo documento, los cuales pueden ser de tipo XML, JSON o BSON principalmente. No es necesario seguir una estructura definida para cada documento, brinda más flexibilidad al momento de almacenar datos con nuevas estructuras.

```
{
  "id" : ObjectId<"57576f554778be749924ce3c">,
  "cliente" : {
    "tipo" : "Regular",
    "apellido" : "Carvajal",
    "nombre" : "John Alexander",
    "ciudad" : "Cali",
    "cuit" : 1324012234
  },
  "tipodePago" : "contado",
  "fechaEmision" : ISODate<"2016-01-22T16:31:10Z">,
  "iten" : [
    {
      "album" : "Peace Sells...But Who's Buying?",
      "año" : 1986,
      "cantidad" : 1,
      "precio" : 35000,
      "artista" : "Megadeth"
    },
    {
      "album" : "Black Sabbath",
      "año" : 1970,
      "cantidad" : 1,
      "precio" : 45000,
      "artista" : "Black Sabbath"
    }
  ],
  "nroFactura" : "DEP110",
  "intereses" : [
    "Metal",
    "Heavy Metal",
    "Tras Metal"
  ]
}
```

```
{
  "id" : ObjectId<"57576f6a4778be749924ce3d">,
  "cliente" : {
    "tipo" : "Regular",
    "apellido" : "Kudlow",
    "nombre" : "Steve",
    "ciudad" : "Cali",
    "cuit" : 4565543090
  },
  "tipodePago" : "TarjetaCredito",
  "fechaEmision" : ISODate<"2016-03-12T16:31:10Z">,
  "iten" : [
    {
      "album" : "Metal on Metal",
      "año" : 1982,
      "cantidad" : 1,
      "precio" : 65000,
      "artista" : "Anvil"
    }
  ],
  "nroFactura" : "DEP300"
}
```

Grafo: Almacena datos relacionados bajo muchos niveles, representaciones de amigos de mis amigos o productos que relacionan a distintos clientes de distintas nacionalidades.

Dos elementos distintivos:

- Los nodos, cuya finalidad es representar las entidades (personas, productos) con sus respectivas propiedades (cuit, nombre)
- Relaciones o Aristas, cuya finalidad será representar tanto el tipo como la dirección de nuestra relación (Gustos, si amigo o no, ubicación) con sus propiedades.



Columnar: dentro de una fila podemos encontrar distintas columnas relacionadas a un Key y a su vez cada columna contiene un par (Key-Value) que hace referencia al campo y su valor.

La ventaja es que no es necesario que todas las filas tengan la misma columna ni la misma cantidad de columnas, permitiendo adicionar.



CLASIFICACIÓN DE LOS DBMS SEGÚN:

- MODELO DE DATOS
 - ❖ RDBMS
 - ❖ OODBMS
 - ❖ NoSql
- NUMERO DE USUARIOS
 - ❖ Monousuario
 - ❖ Multiusuario
- NÚMERO DE SITIOS
 - ❖ Centralizado
 - ❖ Distribuido
 - Distribuido homogéneo
 - Distribuido multibase de datos
- COSTO
 - ❖ Gratuitos
 - ❖ Versiones de prueba
 - ❖ Versiones personales
 - ❖ Venta modular
 - ❖ Limitando usuario o no

- ❖ Monousuarios
- PROPÓSITO
 - ❖ General
 - ❖ Especial

MODELO RELACIONAL

Puntos de análisis para estudiar el Modelo Relacional:

1. Estructura: relaciones, propiedades, tuplas, atributos, dominio, claves, etc
Características o propiedades de las relaciones:
 - a) Las tuplas no están ordenadas -conjunto matemático-
 - b) Los atributos en las tuplas no están ordenados -conjunto matemático-
 - c) Los valores en los atributos deben ser atómicos - relaciones normalizadas-
 - d) No hay tuplas repetidas -hechos diferentes del mundo real-Restricciones (constraints) en valores:
 - a) Implícitas o basadas en el modelo -características o propiedades de las relaciones-
 - b) Explícitas o basadas en el esquema -DDL-
 - Valores atómicos
 - Condiciones de clave:
 - Unicidad: no puede haber tuplas con el mismo valor en atributos que la forman
 - Minimalidad: debe ser el menor conjunto de atributos posible (superclave - clave)
 - c) Reglas de negocio -triggers, aserciones, aplicaciones-
2. Integridad referencial: la base de datos no debe contener valores de FK sin concordancia. Son reglas de integridad generales que están en todas las bases de datos relacionales (Regla de Integridad de Entidades [clave primaria no nula] y Regla de Integridad Referencial [clave foránea debe tener valores de la clave primaria o null])
3. Manipulación de datos: Álgebra relacional
El álgebra relacional proporciona un fundamento formal para las operaciones del modelo relacional. Se utiliza como base para la implementación y optimización de consultas en los RDBMS. Está formada por un conjunto de operadores que operan sobre una o dos relaciones produciendo como resultado una relación -propiedad de cerradura-

ALGEBRA RELACIONAL

- Operadores tradicionales
 - ★ Unión
 - ★ Intersección
 - ★ Diferencia
 - ★ Producto cartesiano

- Operadores relacionales especiales
 - ★ Selección
 - ★ Proyección
 - ★ Reunión
 - ★ División

UNIÓN

Es una operación binaria, conmutativa, asociativa y las cabeceras de las tablas deben ser idénticas, es decir, deben tener el mismo dominio.

El resultado de esta operación es una relación que incluye todas las tuplas que están en cada tabla. Las duplicadas se eliminan.

Estudiantes $\cup$ Docentes

INTERSECCIÓN

Es una operación binaria, conmutativa, asociativa y las cabeceras de las tablas deben ser idénticas, es decir, deben tener el mismo dominio.

El resultado de esta operación es una relación que incluye las tuplas comunes entre ambas tablas.

Docentes $\cap$ NoDocentes

Diferencia

Es una operación binaria, no conmutativa, asociativa y las cabeceras de las tablas deben ser idénticas, es decir, deben tener el mismo dominio.

El resultado de esta operación es una relación que incluye todas las tuplas que están en Docentes pero no en No Docentes.

Docentes – NoDocentes

PRODUCTO CARTESIANO

Es una operación binaria, no conmutativa, asociativa y las cabeceras de las tablas deben ser disjuntas, es decir, no deben tener el mismo dominio.

El resultado de esta operación es una relación donde se combinan todas las columnas y tuplas de las tablas que participan de la operación.

Materias × Pr ovincias

SELECT

Es una operación unaria, aplica restricción a la relación utilizada y tiene operadores lógicos (AND, NOT, OR)

El resultado es una tabla que solo contenga los atributos seleccionados.

$$\sigma_{(Dno=4 \text{ AND } Sueldo>25000) \text{ OR } (Dno=5 \text{ AND } Sueldo>30000)}(EMPLEADO)$$

PROYECCIÓN

Es una operación unaria para elegir columnas.

El resultado de la operación es una tabla que solo contenga los atributos especificados.

$$\pi_{Apellido, Nombres}(NoDocentes)$$

JOIN - REUNIÓN

Es una operación binaria que utiliza el atributo común entre ambas relaciones.

El resultado de esta operación es: en la cabecera se mantiene el atributo común, en el cuerpo se combina según la condición.

<i>Localidades</i>	$\bowtie$	<i>Pr ovincias</i>
<i>Id Pr ovincia = Id Pr ovincia</i>		

- EQUIJOIN (igualdad)

- JOIN NATURAL: * (elimina atributo superfluo en equijoin)-necesita que atributo común tenga el mismo nombre

Reunión Externa- OUTER JOIN – Concatenación Externa



Estudiantes ⋈ *Localidades*
 $IdLocalidad = IdLocalidad$

Localidades ⋈ *Provincias*
 $IdProvincia = IdProvincia$

Estudiantes ⋈ *Localidades*
 $IdLocalidad = IdLocalidad$

DIVISIÓN

Es una operación binaria que compara atributos del divisor incluidos en la cabecera del dividendo.

A					B
u	v	w			w
u2	v4	w1			w1
u1	v5	w5			w3
u1	v7	w4			w4
u2	v4	w3			w5
u3	v1	w5			
u1	v2	w3			
u2	v4	w5			
u3	v2	w1			
u2	v4	w4			

El resultado de $A \div B$ será:

RENOMBRAR

El renombramiento es temporal, solo para las operaciones subsiguientes.

$\rho_S(A)$ Renombra a la relación A como S.

$\rho_{(a,b,c)}(A)$ Renombra a los atributos de la relación A como a, b, c.

$\rho_{S(a,b,c)}(A)$ Renombra a la relación A como S y a sus atributos como a, b, c.

OPERACIONES RELACIONALES ADICIONALES

1. PROYECCIÓN GENERALIZADA: para incluir constantes o funciones sobre atributos

INFORME. $\leftarrow \rho(\text{Dni, SalarioNeto, Gratificaciones, Impuestos})$
 $(\pi_{\text{Dni, Sueldo} - \text{Deducción, } 2000 * \text{Antigüedad, } 0.25 * \text{Sueldo}}(\text{EMPLEADO})).$

2. FUNCIONES DE AGREGACIÓN: SUM, AVG, COUNT, MIN, MAX

$\tau_{\text{COUNT}(\text{Legajo}), \text{AVG}(\text{Nota})}(\text{Exámenes})$

3. AGRUPAMIENTO: para aplicar funciones de agregación a grupos de tuplas

$\tau_{\text{AVG}(\text{Nota})}(\text{Exámenes})$						
<u>IdMateria</u>	<u>Legajo</u>	<u>Turno</u>	<u>Año</u>	<u>Fecha</u>	<u>Nota</u>	<u>LegDoc</u>
207	11904	2	2010	08/02/10	2	20871
306	12802	1	2010	05/02/10	10	22654
459	13789	10	2009	22/12/09	8	21823
333	13789	2	2010	10/02/10	7	21987
290	19098	8	2009	27/11/09	5	21722
306	19098	1	2010	05/02/10	4	22654
333	19098	2	2010	10/02/10	7	21987
207	19098	2	2010	08/02/10	3	20871
333	24561	10	2009	21/12/09	8	21987

Grupo 1-Legajo=11904-AVG= 2

Grupo 2-Legajo=12802-AVG= 10

Grupo 3-Legajo=13789-AVG= 7.5

Grupo 4-Legajo=19098-AVG= 4.75

Grupo 5-Legajo=24561-AVG= 8

RELACIONES TEMPORALES

Permiten subdividir operaciones complejas y anidadas.

$$temp1 \leftarrow (\tau_{Legajo, AVG(Nota)}(Exámenes))$$

$$temp2 \leftarrow (temp1 \bowtie_{Legajo=Legajo}(Estudiantes))$$

$$temp3 \leftarrow (\pi_{Legajo, Apellido, Nombres, AVG_Nota}(temp2))$$